INF1350
Sistemas Reativos

Comunicação HTTP

Prof.: Adriano Branco abranco@inf.puc-rio.br

Slides baseado em aulas anteriores da Prof. Noemi Rodriguez

alguns padrões

- **cliente-servidor**
- pipeline ou filtros
- p2p

API de sockets

- mensagens são sequência de bytes
- enderecamento por (IP, porta)
- envio (quase sempre) assíncrono e recebimento (quase sempre!!!) síncrono

http no nodemcu

- biblioteca cliente
  - operações para `get`, `post` e `put`
- para servidor
  - uso direto de API sockets

protocolo HTTP

- protocolo textual bastante simples

```
GET /favicon.ico HTTP/1.1
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.14; rv:76.0)
Accept: image/webp,*/*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
...
```

protocolo HTTP

- cliente abre e fecha uma conexão TCP a cada pedido
- mensagens são strings
- servidor tipicamente responde com string HTML
- uso típico da API de sockets

servidor TCP no NodeMCU

```lua
local srv = net.createServer(net.TCP)

local function receiver(sck, request)
    ...
end

if srv then
    -- escuta na porta padrão do HTTP:
    srv:listen(80, function(conn)
        print("cliente conectado!")
        conn:on("receive", receiver)
      end)
end
```

protocolo HTTP - exemplo

```
local function receiver(sck, request)
    print("recebeu:\n" .. request)
    local buf = [[
    <html>
    <body>
    <h2><u>INF1350 - PUC Rio</u></h2>
    <h3><i>ESP8266 Web Server</i></h3>
    <p> boa tarde! </p>
    </body>
    </html>
    ]]
    sck:send(buf,
             function() -- callback: fecha o socket qdo acabar de enviar
               print("respondeu")
               sck:close()
             end)
end
```

HTML: links

- marcação para links:

```
<a href="?pin=LIGA1"><button><b>ON</b></button></a>
```

HTML: links

- marcação para links:

```
<a href="?pin=LIGA1"><button><b>ON</b></button></a>
```

- aparecerá na requisição como:

```
GET /?pin=LIGA1 HTTP/1.1
```

HTML: links

- marcação para links:

```
<a href="?pin=LIGA1"><button><b>ON</b></button></a>
```

- aparecerá na requisição como:

```
GET /?pin=LIGA1 HTTP/1.1
```

- servidor/atuador no NodeMCU:

  https://www.inf.puc-rio.br/~abranco/sr-23.2/code/nodemcu/comm/servidorpisca.lua

AP WIFI + Servidor HTPP

- Executar primeiro `ap.lua`, não esqueça de alterar o `ssid` para incluir o ID da sua placa.

  https://www.inf.puc-rio.br/~abranco/sr-23.2/code/nodemcu/comm/ap.lua

- Depois executar o servidor/atuador no NodeMCU.

  https://www.inf.puc-rio.br/~abranco/sr-23.2/code/nodemcu/comm/servidorpisca.lua

- Conectar na rede wifi - `ssid` que você configurou no `ap.lua` (`nodemcu`*`XX`*). Pode ser com seu smartphone.
- Abrir o navegador e digitar o `ip` configurado no `ap.lua` (`192.168.0.20`)

Mini Projeto NodeMCU

- código usando todos os componentes da placa
  - https://www.inf.puc-rio.br/~abranco/sr-23.2/code/nodemcu/demopinos.lua
- http e mqtt
  - cuidado com https!
- uso de geolocalização
  - `wifi.sta.getap(listap)`
- programa pode envolver apenas NodeMCU ou várias partes
  - prévia de projeto final?